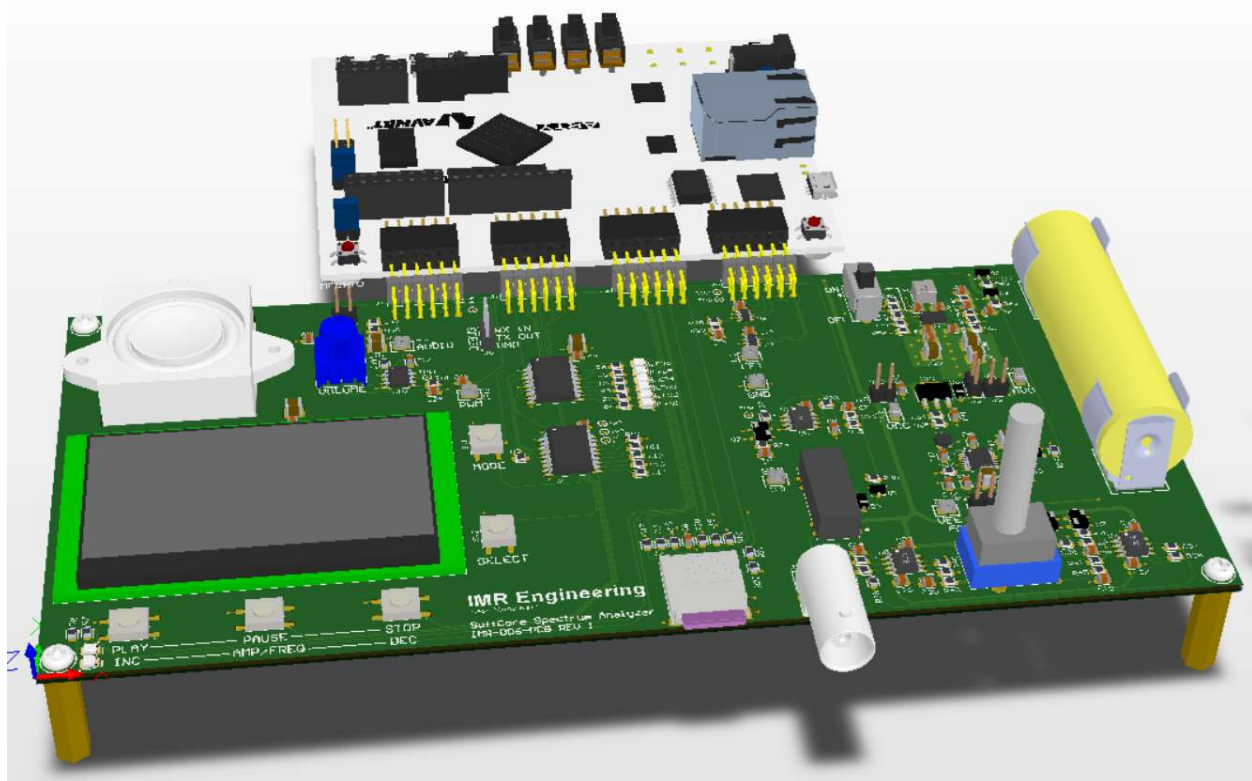


SOFTCORE SPECTRUM ANALYZER

IMR Engineering, LLC

Ideas Made Real



USER MANUAL AND TECHNICAL SPECIFICATION

Principal Engineer: Hab Collector

<http://imrengineering.com>

Table of Contents

1. INTRODUCTION:	4
1.1 THE CHALLENGE:	4
1.2 PHASE 1 / PHASE 2:	4
1.3 SO WHAT IS IT:	4
2. AT A GLANCE:	5
2.1 PL FEATURES:	5
2.2 HW FEATURES:	5
2.3 FW FEATURES:	5
2.4 SIMPLIFIED BLOCK DIAGRAM	6
3. PROJECT SCOPE:	6
3.1 P1 FIRMWARE SCOPE:	6
3.2 P1 HARDWARE SCOPE:	6
4. DETAILED MODES OF OPERATION:	7
4.1 AUDIO SA MODE DETAILS:	7
4.2 SIGNAL SA MODE DETAILS:	9
5. CUSTOM RTL POOR MAN'S DMA:	10
6. FFT IMPLEMENTATION AND NEED TO KNOW:	12
6.1 FFT PROCESS FLOW:	12
6.2 WHAT CAN BE MEASURED:	12
6.3 SIZE BINS AND RBW:	14
6.4 WINDOW FUNCTION:	15
6.5 SAMPLE JITTER – THE PHASE ERROR:	15
6.6 HARMONICS:	16
7. USER INTERFACE:	17
7.1 CONTROLS	17
7.2 DEBUG PORT:	18
8. BOOT LOADER:	18
9. PHASE 2 LOOK AHEAD:	19
9.1 ENHANCEMENTS:	19
9.2 CORRECTIONS:	20
10. TECHNICAL SPECIFICATIONS:	21
10.1 ABSOLUTE MAXIMUM RATINGS	21

10.2	OPERATIONAL RATINGS:.....	21
11.	TIMING.....	22
11.1	IO EXPANDERS:	22
11.2	DISPLAY (SSD1309 CHIP SET):.....	23
11.3	USD DATA TRANSFER	23
11.4	ANTI-ALIASING FILTER DISCONTINUITY AT THE 0V:	24
12.	ACKNOWLEDGEMENT	26

DOCUMENT REVISION HISTORY		
DATE	DESCRIPTION	EDIT BY
2026-03-06	Initial draft - Preliminary	HSC
2026-03-14	Updated formatting and sample jitter	HSC
2026-03-23	Final edits before publishing P1 content	HSC
2026-04-26	Added poor man's DMA, changes to acknowledgement	HSC

1. INTRODUCTION:

1.1 THE CHALLENGE:

For many applications the microcontroller (μ C) architecture provides an excellent balance of performance, cost, and manageable development complexity. It is therefore natural to ask an important engineering question: where does a pure Processor Logic (PL) based FPGA system fit into this landscape?

In comparison there is nothing easy, cheap or quick about an FPGA. The FPGA simply cannot compete in those categories, but for everything else it excels. The challenge explored in this project is not a feature-for-feature comparison between an FPGA and a μ C. Instead, the goal is to explore the practical capabilities of a pure PL FPGA platform by implementing two distinct design projects and observing how such a system compares in practice to what might traditionally be implemented on an ARM Cortex M4/M7 class device or higher.

The first phase (P1) of this exploration focuses on constructing an FPGA-based system that resembles the functional architecture commonly found in a μ C environment. Within the PL, a subset of needed peripherals are implemented which include: UART, SPI, DDR3, Timers, PWM, Interrupt Controller, FPU, Cache memory and other system services required to support a complete embedded application. The resulting system demonstrates that the fundamental building blocks normally associated with a μ C can be tailored specifically to create the device needed. That said, incorporated into this first phase a preview of what lies ahead through the inclusion of two custom RTL-based IP blocks (aka modules) – something you cannot do within a μ C along with a few other FPGA differentiators.

1.2 PHASE 1 / PHASE 2:

What is covered here is the P1 project design: PL, Hardware, and Firmware. In doing so I will show performance and limitations thereof, things I got wrong, and issues in hardware, firmware, and PL I intend to correct, and or, address in Phase 2.

1.3 SO WHAT IS IT:

A dual mode Spectrum Analyzer (SA) implemented around the MicroBlaze Softcore IP processor running on the AMD (aka Xilinx) XC7A100T-1CSG324-1 FPGA. As this FPGA is pure PL there is no PS (Processor System) as in the case of SoC based FPGAs. The system architecture recreates many of the services typically found in a microcontroller environment while remaining entirely within the FPGA PL fabric. The user interface is intentionally simple, and hardware driven, consisting of five multi-function push buttons and a monochrome 128x64 graphical display used for system control and visualization. The device operates as a compact, self-contained instrument designed to demonstrate real-time spectral analysis. The project currently supports two modes of operation: Audio SA (referenced in FW as Audio_SA) and Signal SA (referenced in FW as Signal_SA). Audio Mode displays the spectral content and signal strength of a WAV audio file played from μ SD. Signal Mode mimics the behavior of a desktop Spectrum Analyzer.

2. AT A GLANCE:

2.1 PL FEATURES:

- MicroBlaze Softcore:
 - 64KB / 32KB of Instruction / Data Cache, FPU and 128KB of LBM
- 3x Timers (two with interrupts)
- PWM
- 1x SPI shared to 3x devices: 2x IO Expanders and 1x SSD1309 Display
- 1x SPI dedicated to μ SDB
- 1x SPI dedicated to 16MB QUAD SPI Boot Flash – Bootloader use only
- UART
- DDR3 Interface to external 128Mx16 (MT41K128M16XX-15E)
- 11x GPIO Outputs
- 12x GPIO Inputs
- Custom RTL Revision IP
- Custom RTL ADC Interface IP
- Interrupt Controller with 3x Fast Interrupts
- System Bus Interconnect

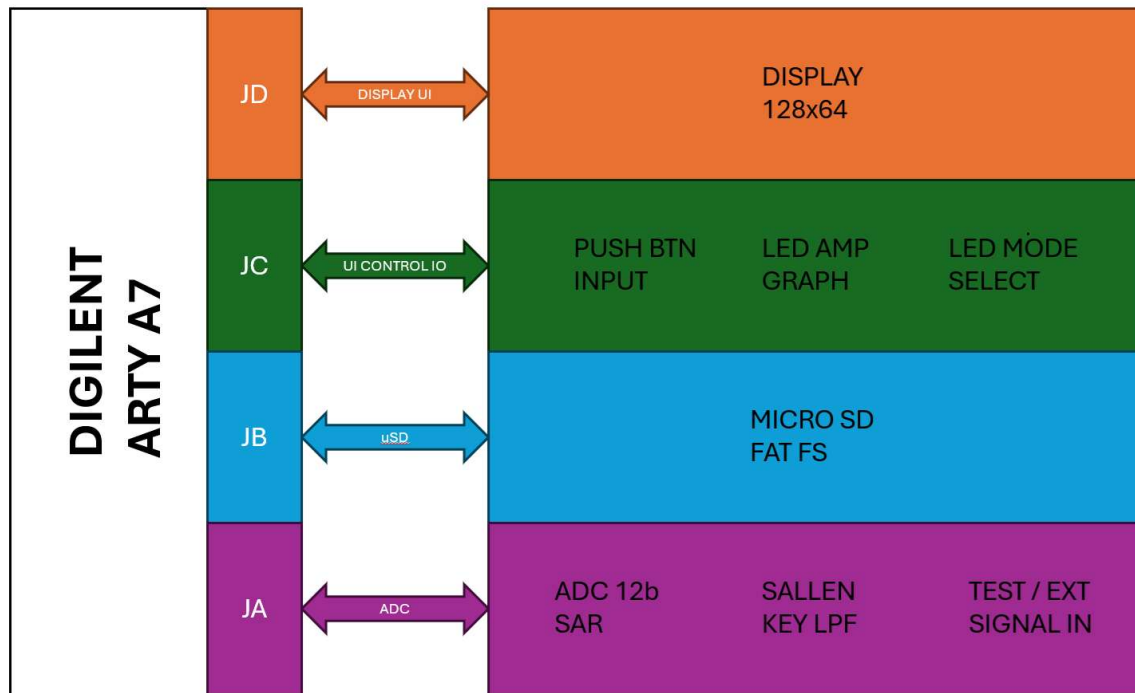
2.2 HW FEATURES:

- IO Expander to LED outputs
- IO Expander to UI push button Inputs
- Micro-SD
- 128x64 2.42" OLED Display
- Dual 12b 1MSPS ADC
- Anti-Aliasing Sallen Key LPF
- Selectable Analog inputs to Signal SA
- Local Variable Oscillator 4.4K to 48.2KHz
- Class AB Audio Amplifier
- Single Li-ION cell operation
- Interface to Digilent Arty A7 Development board

2.3 FW FEATURES:

- Bear Metal Application
- Middleware: FAT FS Library
- Middleware: U8G2 Graphics Library
- Middleware: KISS FFT Library
- Two modes of operation: Audio and Signal

2.4 SIMPLIFIED BLOCK DIAGRAM



3. PROJECT SCOPE:

When creating a two-part project of such hardware and firmware complexity, as a one-person show, limiting scope is necessary to ensure time, budget and project momentum are kept.

3.1 P1 FIRMWARE SCOPE:

For firmware I decided to implement a bear-metal application rather than an RTOS. It limited FW complexity for one, but it also availed the processor of the memory and processing time needed to support an RTOS. The choice of KISS FFT pre-existing library was also necessary as a custom FFT library would be a project on to itself.

3.2 P1 HARDWARE SCOPE:

The complexity of layout for a DDR3 memory and the layer count associated with such a design from a cost and time perspective made the choice of using the off-the-shelf Digilent Arty A7 Development Board an easy one. Having designed both FPGAs and DDR3 memories before I knew ahead of time I was looking at 3 revisions of a 6 layer PCB. More so, I have chosen not to extend my Altium license moving forward, the need for trace impedances and serpentine routing neither of which is supported by Circuit Studio (development tool used). For the same reasons the choice of a simple display was made over an RGB color model.

4. DETAILED MODES OF OPERATION:

- Audio Spectrum
- Signal Spectrum

4.1 AUDIO SA MODE DETAILS:



In Audio SA mode the system operates as a real-time audio playback and spectral display instrument. Audio content is read from a WAV file stored on the μ SD card using the FAT FS middleware. Once the file header is parsed and verified, audio sample data is moved from storage into a large streaming circular buffer sized at 22K samples / 44KB (int16_t data). This large, main buffer exists to reduce the frequency of μ SD card read operations and to provide margin against storage access latency. For this streaming buffer the system fills a smaller, secondary 4KB feed buffer which serves to replenish the larger buffer. In this way the buffering scheme is staged: mass storage feeds the main streaming buffer on start, thereafter the main buffer is feed from the secondary buffer as space becomes available, as the main buffer supports both playback and FFT processing (FFT is processed by the KISS FFT middleware). The buffer is sized to support a max audio sample of 22050Hz playback. It could have been sized larger, but I choose to limit the size here, with planned use of that memory in phase 2. Additionally, six colored LEDs (arranged as a bar) display the magnitude (signal strength no signal spectrum) of the playback from -30dB to -10dB in -4dB increments. Both the spectral display and the signal strength LED bar will provide a visual interpretation of the playback audio.

Playback timing is controlled by a timer interrupt service routine (ISR) configured to run at the audio playback sample rate. For a playback rate of 16KHz, the ISR period is:

$$T_{sample} = \frac{1}{16000} = 62.5 \mu s$$

This means one audio sample is processed every 62.5 μ s. As playback proceeds, samples are simultaneously routed to the PWM audio output path and copied into the FFT sample buffer. The FFT sample buffer contains 1024 samples, therefore the time required to fully populate at 16KHz playback is:

$$T_{FFT\ Fill} = \frac{N}{F_s} = \left(\frac{1024}{16000} \right) = 64\ ms$$

Thus, under continuous playback, a new 1024-point FFT frame becomes available every 64ms. This corresponds to an FFT update rate of:

$$\frac{1}{0.064} = 15.625 \text{ Hz}$$

or approximately 15.6Hz spectral updates per second. As an additional reference, the 4KB feed buffer holds 4096 bytes. For 16-bit PCM audio, each sample is 2 bytes, giving:

$$\frac{4096}{2} = 2048 \text{ samples}$$

At 16KHz playback this secondary buffer represents:

$$\frac{2048}{16000} = 128 \text{ ms}$$

of mono audio data. Likewise, the 44KB streaming buffer holds:

$$\frac{44056}{2} = 22528 \text{ samples}$$

which at 16KHz corresponds to:

$$\frac{22528}{16000} = 1.408 \text{ s}$$

or about 1.4 seconds of mono playback data. These staged buffer depths are what allow playback, storage access, and FFT processing to coexist with acceptable continuity.

Interrupt handling is a significant part of the design – its importance cannot be overstated for this project. As such I take a moment to expand upon it here. Both modes rely heavily on the sample rate interrupt timer (AXI Timer 1). The MicroBlaze is capable of normal and low latency (fast) interrupts. Normal interrupts use the AXI interrupt controller and are decoded, prioritized and vectored to the processor. Normal interrupts take about 40 cycles between detection to ISR jump. Fast interrupts bypass the interrupt controller and connect directly to the MicroBlaze. Subsequently fast interrupts are processed within about 10 cycles, generally speaking, on par with ARM Cortex M4/M7. That said, most M4 and M7 designs clock in excess of 100MHz, so the actual time from detection to ISR jump and ISR processing is less. Part of the interrupt mechanism is architectural, and part is simply clock speed (cycle time): The P1 design runs at 100MHz. In P2 I will attempt to run the same design to 150MHz. A difference with a distinction. While μ Cs are typically specified with a fixed max clock speed, FPGAs come in speed grades -1, -2, -3 (slow, faster, fastest). Generally speaking, the speed grade represents a “possible” range of the max fabric clock frequency. At speed grade 1 it maybe possible to get a max of 200MHz. If this is repeatable across devices will be device and design dependent. That said, 100MHz is just a known starting point. Nesting, the MicroBlaze interrupt controller does not support a true nested interrupt model where one interrupt preempts another already in service, though it does support interrupt priority (fixed in the PL design). In this project that reality strongly influenced the overall firmware architecture, the selected FFT size, and the balance between playback, analysis, and display update rate. Lastly, all ISR are placed in (Local Block Memory) LBM (FPGA block RAM directly attached to the MicroBlaze – runs exceptionally fast) and not within DDR3 space. The overall size of the application dictates that most of it runs from DDR space, targeting LBM for specific use

where speed is paramount. Another distinction here is the FPGA application runs (mostly) from DDR and LBM memory, whereas with the μC the application runs from FLASH. The FPGA requires a far more complex linker – the reader is encouraged to review the application linker file (lscript.ld) with the Virtual Address Memory (VAM) and Local Address Memory (LAM) and how its loaded at run-time.

4.2 SIGNAL SA MODE DETAILS:



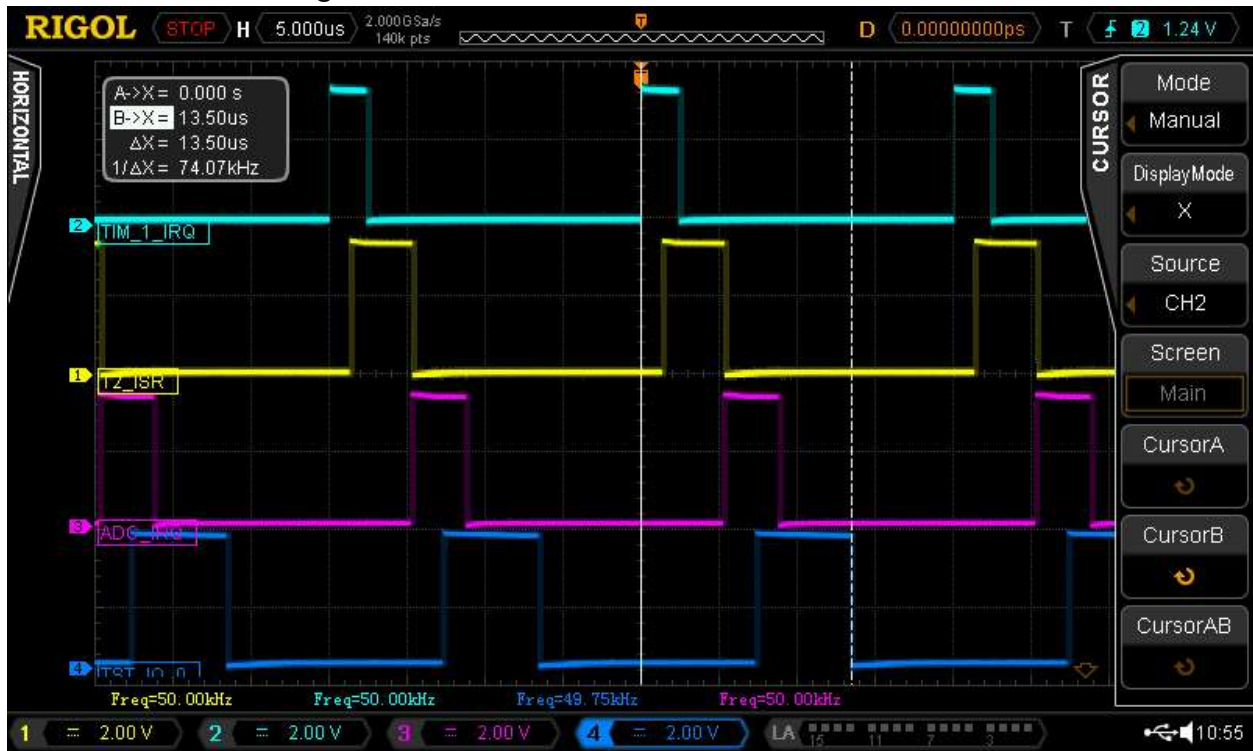
In this configuration the instrument functions as a traditional Spectrum Analyzer by converting incoming time-domain signals into its frequency-domain representation. While the sampling data differs from Audio SA mode, the spectral analysis engine remains fundamentally the same. Both modes leverage the same FFT algorithm and the same 1024 samples per frame,

Signal acquisition is a two interrupt event. It begins with the sample-rate timer interrupt. Each timer interrupt event initiates the sampling sequence by triggering the custom ADC RTL IP to perform an ADC conversion. The ADC IP implemented in PL controls the timing of the conversion and provides the resulting digital sample to the firmware via a fast interrupt (the second). Within the ISR the sample is transferred into the FFT sample buffer, where samples are accumulated until the frame buffer (1024 samples) is full.

Once the frame is complete, the spectral processing routine is executed. During this time the Timer ISR is suspended. The resulting magnitude values represent the spectral energy distribution of the input signal and are mapped to the display. Once the display has been updated the ISR is resumed.

Input signals can originate from two sources. The onboard, variable Wien Bridge Oscillator (aka local oscillator - 4.4KHz to 48.2KHz) or via an external BNC connection. As with Audio SA mode, the entire signal acquisition and analysis pipeline is driven by the timer interrupt architecture. The interrupt establishes the sampling cadence, triggers ADC conversion, and governs the rate at which samples are collected for FFT processing. Consequently, the overall performance and responsiveness of the Signal SA system are fundamentally driven and limited by the ISR-based architecture.

Below shows the timing interactions:



NOTES:

CH2 - TIM_1_IRQ: Timer 1 IP IRQ

CH1 - T2_ISR: Timer 1 ISR handler (initiates Custom RTL ADC conversion)

CH3 - ADC_IRQ: Custom RTL ADC conversion complete IRQ

CH4 - TEST_IO_0: ADC IRQ ISR complete (the sample has been captured)

Cursor Marking: Shows the max frequency possible for sampling about 74KHz – choose 50KHz for sampling frequency.

As the above timing analysis shows the max sample rate possible using this architecture resolves to an absolute maximum of 70KHz (allowing some wiggle room from the 74KHz). With the 100KHz out of reach for phase 1 I settled on a sample rate of 50KHz, which implies a usable frequency BW of 25KHz. *Note the local Wien Bridge Oscillator variable range was designed to be the intended 100KHz sample rate and 50KHz BW (4.5KHz to 48.2KHz).*

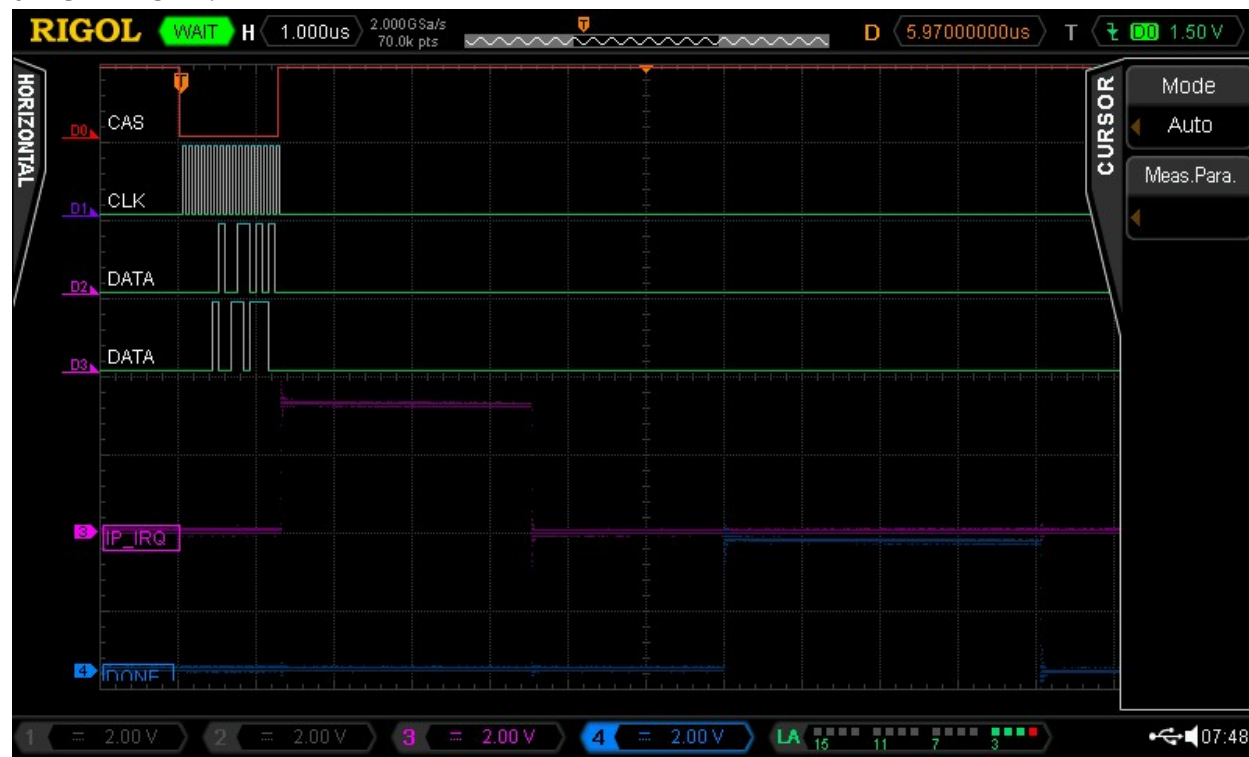
5. CUSTOM RTL POOR MAN'S DMA:

Two custom RTL IPs were developed for this project, each with custom firmware drivers. Notably, a custom ADC acquisition IP (IMR_ADC_7476A_X2_1_0) is memory-mapped on the AXI bus to five registers: Control, Status, ADC Data A, ADC Data B, and IRQ. The IP supports two modes: Single, and Multi. Dual channel simultaneous conversion.

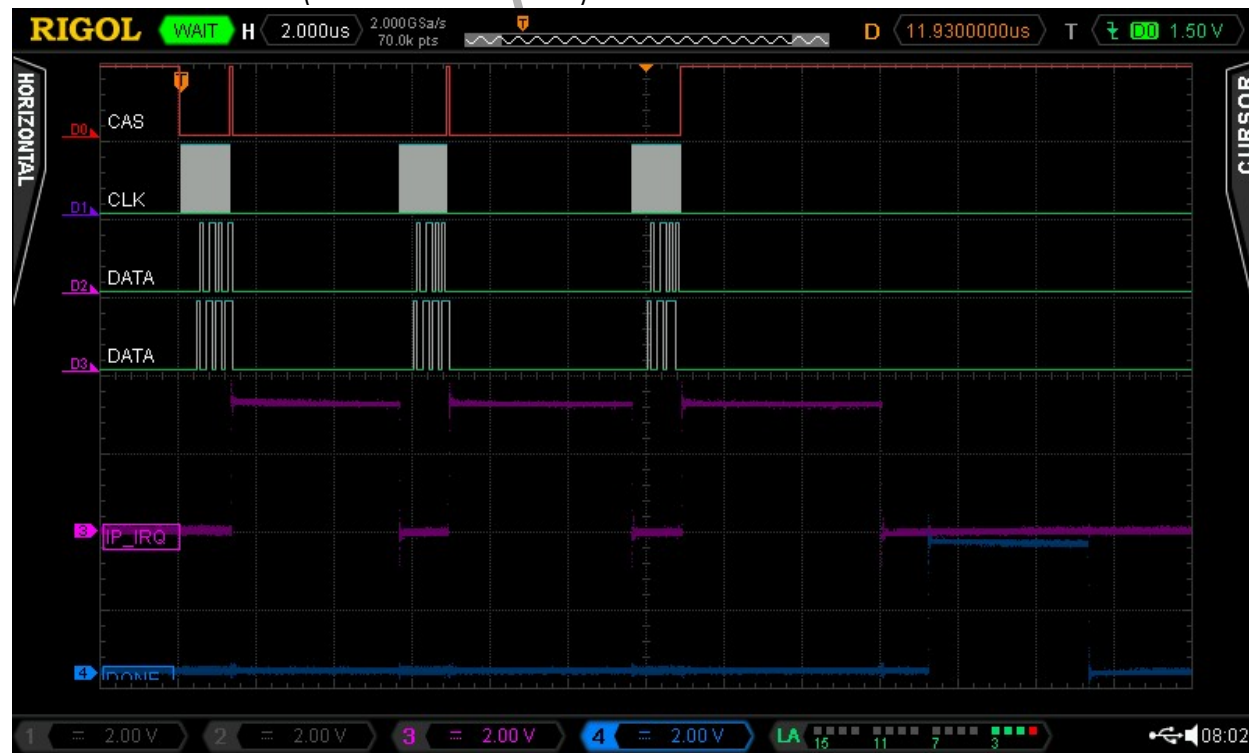
Two IRQs are associated with the IP. The IP IRQ asserts after every ADC acquisition, in both modes. In single mode, the custom driver (AXI_IMR_ADC_7476A_DUAL.c/.h) captures one acquisition; the ISR reads both channels, stores the data to a buffer, then asserts the "Acquisition Done IO" line, which feeds the IRQ Controller and generates a second interrupt. This second Done interrupt is ignored in single mode. In Multi-Conversion Mode, the driver captures N samples; the IP IRQ asserts after each acquisition and increments

the buffer pointer. After N samples, the Done IO line asserts, effectively creating a “poor man's DMA” with a dedicated Done IRQ to indicate memory capture is complete.

SINGLE MODE:



MULTI CONVERSION (POOR MAN'S DMA) MODE:



6. FFT IMPLEMENTATION AND NEED TO KNOW:

6.1 FFT PROCESS FLOW:

In essence the FFT categorizes the sampled time-domain signal into spectral energy between the baseband and the Nyquist. Because the FFT works on a finite number of samples (frame), the result is always tied to one snapshot in time. In this design, that snapshot is 1024 samples. Every time that frame becomes available the FFT is processed and can subsequently be displayed.

6.2 WHAT CAN BE MEASURED:

The system takes 50,000 equally spaced samples every second. That sample rate creates a hard upper limit on what can be correctly processed by the FFT. As the Nyquist is 25KHz, spectral content above 25KHz represents an aliasing error. Higher frequency content feeding into the ADC folds back into the measure band and appears as an incorrect lower frequency – the aliasing effect. The intent was for the anti-aliasing filter to reject said frequencies. That said, the ADC front end Sallen-Key LPF was designed for this purpose with a BW of 330KHz. At the time, the expectation was that the local Wien Bridge Oscillator would be “clean” and the 330KHz (-40dB/decade roll-off) would allow for some limited Phase 2 development on the same Phase 1 hardware platform. I was wrong about the local oscillator being clean. Another thing I failed to realize at the time was that harmonic content beyond the BW limit would also alias back. The aliasing error follows the form:

$$Fa = \text{abs}(Fin - kFs)$$

Fa: Alias frequency

Fin: Input frequency

Fs: Sample frequency

k: Integer multiple until *Fa* falls in the Nyquist band

Observed Example of Alias Content 1:

$Fin = 120\text{KHz}$

$\text{Abs}(120\text{KHz} - (1)(50\text{KHz})) = 70\text{KHz}$ (not within Nyquist so will not be displayed)

$\text{Abs}(120\text{KHz} - (2)(50\text{KHz})) = \mathbf{20\text{KHz within Nyquist so it will be displayed}}$

Observed Example of Alias Example 2:

$Fin = 90\text{KHz}$

$\text{Abs}(90\text{KHz} - (1)(50\text{KHz})) = 40\text{KHz}$ (not within Nyquist so will not be displayed)

$\text{Abs}(90\text{KHz} - (2)(50\text{KHz})) = \mathbf{10\text{KHz within Nyquist so it will be displayed}}$

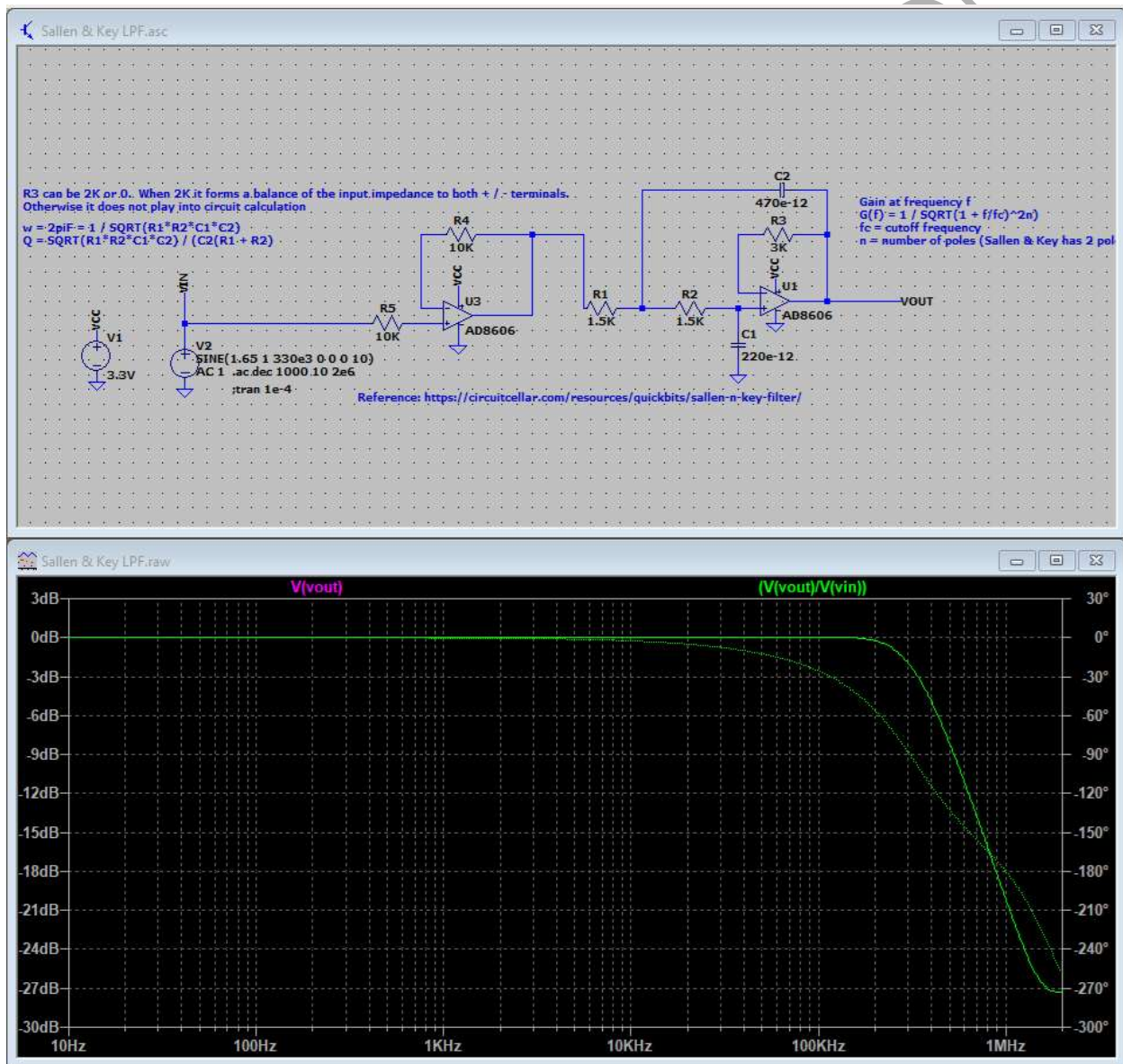
Observed Example of Alias Example 3:

$Fin = 35\text{KHz}$

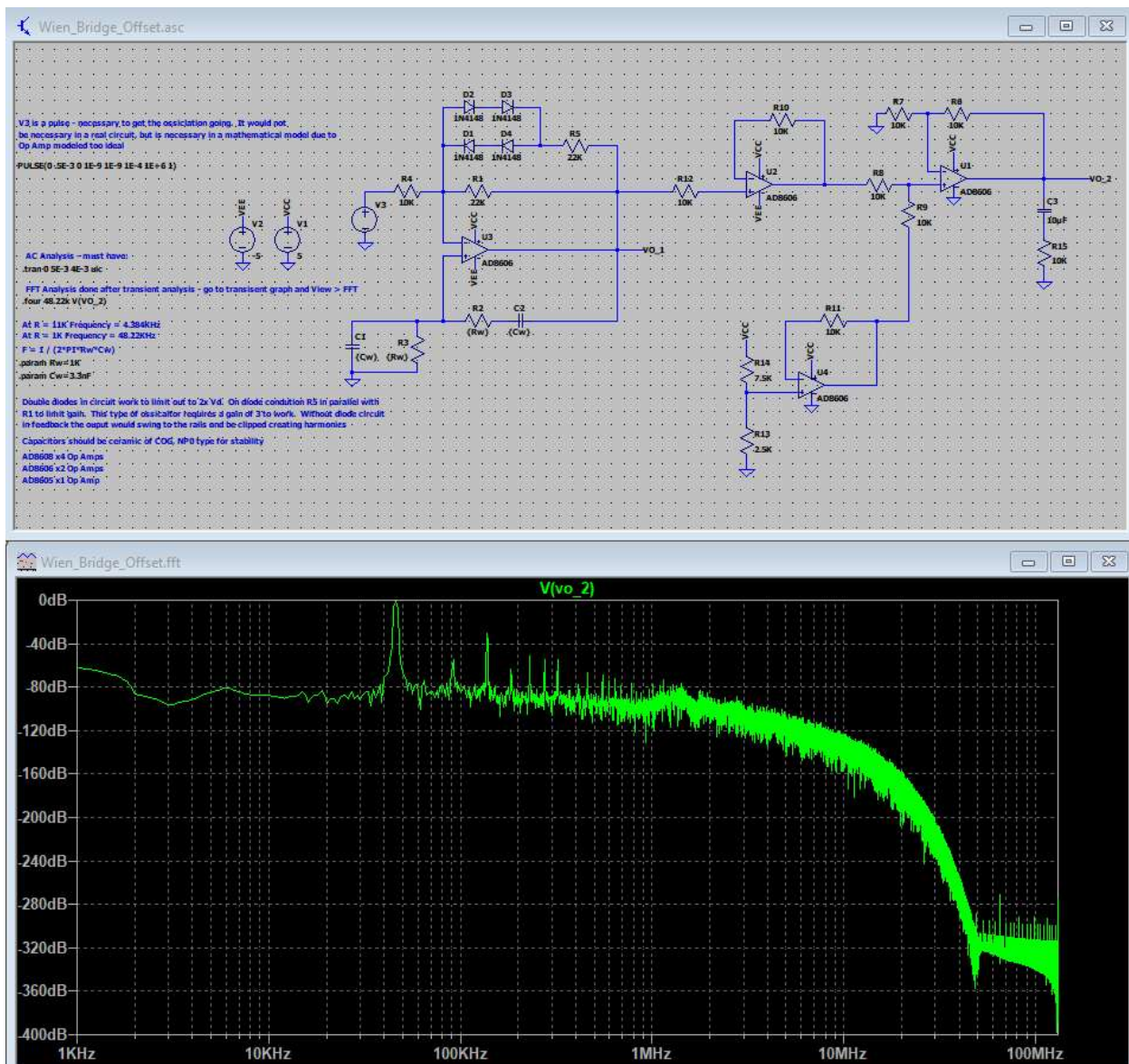
$\text{Abs}(35\text{KHz} - (1)(50\text{KHz})) = \mathbf{15\text{KHz within Nyquist so it will be displayed}}$

Lastly, it is important to understand that the alias frequency is NOT an error of the FFT, but rather a limitation of the front-end anti-aliasing filter, which at its high -3dB limit does not

reject frequencies above the implemented sampling rate. As can be seen from the spectral plots below, the local oscillator created spectral energy beyond both the implemented and (initially) intended sample rate. That energy is believed to have been caused by the clipping diodes in the feedback circuit. This phenomenon was observed with the device operating in Signal Spectrum Mode with the local oscillator tuned within the Nyquist range. As demonstrated below, there was higher-frequency content present in the Wien Bridge oscillator beyond the desired fundamental. Because the anti-aliasing filter cutoff frequency was set too high, those higher-frequency components were allowed to reach the ADC and were subsequently manifested in the displayed spectrum.



Spice Simulation of Anti-Aliasing Filter



Spice simulation showing spectral response of local oscillator (Hanning Window Applied)

6.3 SIZE BINS AND RBW:

FFT size tells you how many samples per frame. In this design the FFT size is 1024 (1K). The FFT output is divided into bins. Each bin represents one narrow frequency slot. For a real-input FFT only the non-negative half of the spectrum is unique, so the useful output consists of bins 0 through FFT_Size / 2 (512), with bins 0 (DC offset) and bin 512 (Max BW) being unique. Bins 513 to 1024 represent the Complex Conjugate (CC) of the first half and subsequently mirror the first half in magnitude. With Bin 513 the CC of bin 511, bin 514 the CC of bin 510, etc. The frequency width of each bin is the Resolution Bandwidth (RBW). It is set by sample rate divided by FFT size. With a 50,000 Hz sample rate and a 1024-point FFT, the RBW for this project is 48.828Hz (per bin). Bin 1 is centered on 48.8 Hz, bin 10, 488.3 Hz, bin 100, 4.883 kHz, and so on. A smaller RBW gives finer spectral detail because bins are narrower, however with a finer RBW comes more processing and with it, slower

updates. Lastly, the DC and low frequency content of the signal is contained in Bin 0 and Bin 1 (50 / 60Hz). As such I deliberately omit bin 0 content from being displayed as it is likely to be quite large and may not be what the user expects to see.

6.4 WINDOW FUNCTION:

FFT computations assumes that the frame repeats indefinitely and in-sync. In other words, each sample index within a frame is assumed to match the corresponding index in the frame before and after it. Stated differently, the FFT assumes there is no discontinuity between frames, and that the signal begins at the same point in each frame.

In practical application this is almost never the case. As such, a "window function" is applied to the sample data. As the frame to frame transition does not repeat, the window function tapers the beginning and end of the sampled data so the FFT does not see a spike / discontinuity / sudden jump when the record repeats. That discontinuity is Frame boundary discontinuity. Additionally, the window function provides a smoothing operation across adjacent bins should a given input frequency not fall on an exact RBW bin interval. For example, with a RBW of 10Hz and an input frequency of 15Hz – the window serves to spread energy across 10Hz and 20Hz bins. There are many Window functions to choose from. This project uses the Hanning Window. Computationally the Hanning Window:

$$Window[Index] = 0.5 \left(1 - \cos \left(\frac{2\pi(Index)}{N - 1} \right) \right)$$

Window[Index] = Window Index Coefficient

N = FFT Size

Index = Sample Index ($0 \leq Index \leq (N - 1)$)

Mathematically the Window function is applied across the frame data as such:

$$FFT_{Data[Index]} = FFT_{SampleData[Index]} \times Window_{Function[Index]}$$

For computational speed the Hanning Window is calculated once at POR.

6.5 SAMPLE JITTER – THE PHASE ERROR:

Sample timing jitter manifests itself in phase noise. The more of less of it present raises or lowers the noise floor of the displayed spectrum and can manifest itself as "jumpiness" in the display. Phase error is fundamentally caused by sample to sample timing variations (jitter), but, and in the case of this P1 application, the effect is more apparent due to the FFT computational processing and display (Frame to Frame).

Unlike Audio SA Mode, in Signal SA Mode the FW is architected such that after each frame is acquired (via ISR timing), the acquisition is paused, the data processed and display updated, then acquisition is resumed. The processing and updating of data (timing delay) creates the phase offset (error) between successive frames. Because this timing offset repeats every frame, it manifests as phase variation in the sampled signal and contributes to visible instability in the displayed spectrum. This architectural limitation will be addressed in Phase 2.

6.6 HARMONICS:

A perfect sine wave contains only one frequency - the fundamental. As perfection is hard to find in real life for everything else there are harmonics. Harmonics are integer multiples of the fundamental. Any nonlinearity in the signal path can reshape the waveform. Clipping, asymmetric biasing, saturation, crossover distortion, and nonlinear amplifiers all impact waveform shape. The FFT reveals that shape change as harmonic energy. Lastly, any given waveform can be a composite of multiple fundamentals.

Odd harmonics are often associated with distortions that preserve waveform symmetry about its midpoint (zero). In this case, the top and bottom halves of the waveform look the same. For example, a sine wave that is clipped equally on both the positive and negative peaks preserves this symmetry. A square wave is the typical example and contains strong odd harmonics. Even harmonics increase when the waveform becomes asymmetric about zero. In other words, the bottom half of the signal does not mirror the top half. One example would be clipping of a sine wave but only on the positive or negative side, not both. That said, real-world signals typically contain both odd and even harmonics. The point here is that the ADC subtraction used to center the signal only removes the DC offset and therefore only impacts Bin 0 of the FFT. It does not alter the harmonic content of the signal.

The practical lesson is simple: when you see harmonic peaks, the FFT is telling you something about waveform shape and system linearity, not just frequency. The fundamental says where the signal is. The harmonic pattern says how clean or distorted that signal is. This phenomenon can be observed with the device in Signal Spectrum Mode. I said all of that to say the anti-aliasing filter created a discontinuity at 0V when using the external signal generator via the BNC connector, but not so when using the internal local oscillator. In either mode, by using persistence on the scope, it can be seen additional frequency content has been added at the filter output, though my desktop SA instrument confirmed this content well beyond the BW of the Signal SA Mode. I suspect, but did not confirm, this is caused from the SAR ADC that it feeds. Note the signal generator has a stated THD of less than -55dBc, from the spice simulation of the local oscillator we see its THD is far more than that. See sections [ANTI-ALIASING FILTER DISCONTINUITY AT THE 0V:](#) and [WHAT CAN BE MEASURED:](#) Something to be addressed and corrected in P2.

The input signal, via the local oscillator or the external BNC has an intended minimum of 0V. This was purposely designed to simplify input protection and to support the intended development flow (implementing features incrementally and verifying operation step by step). In Signal SA Mode, the ADC output is centered digitally. The ADC is 12-bit, so its midpoint occurs at 2048 counts. To create a zero-centered signal, 2048 is subtracted from each sampled value, effectively shifting the midpoint to zero. This results in a signal representation containing both positive and negative values.

Because the analog input signal itself can only be positive, this centering operation also removes the DC offset inherent in the sampling method. However, if the input signal is offset slightly above or below the expected midpoint, the simple subtraction of 2048 will not fully remove the DC component. Any remaining DC content appears in Bin 0 of the FFT. The Audio SA Mode does not suffer from this issue because the audio samples are stored as PCM16 (signed 16-bit) values, which are already centered around zero. In this sense, subtracting the ADC midpoint removes the DC offset associated with the unipolar input range. This operation affects only the DC component (FFT Bin 0) and does not alter the harmonic content of the signal. For this reason I do not display bin 0.

Total Harmonic Distortion (THD) is a measure of how much harmonic energy exists in a signal compared to the fundamental frequency. It quantifies the amount of distortion introduced by nonlinearities in a system. A perfectly linear system producing a pure sine wave would have 0% THD. As perfection is beyond our grasp, as distortion increases, the amplitudes of the harmonic frequencies increase relative to the fundamental. THD is typically expressed as a percentage or in decibels (dB). The general form is:

$$THD(\%) = 100 \times \sqrt{\frac{V_2^2 + V_3^2 + V_n^2}{V_1^2}}$$

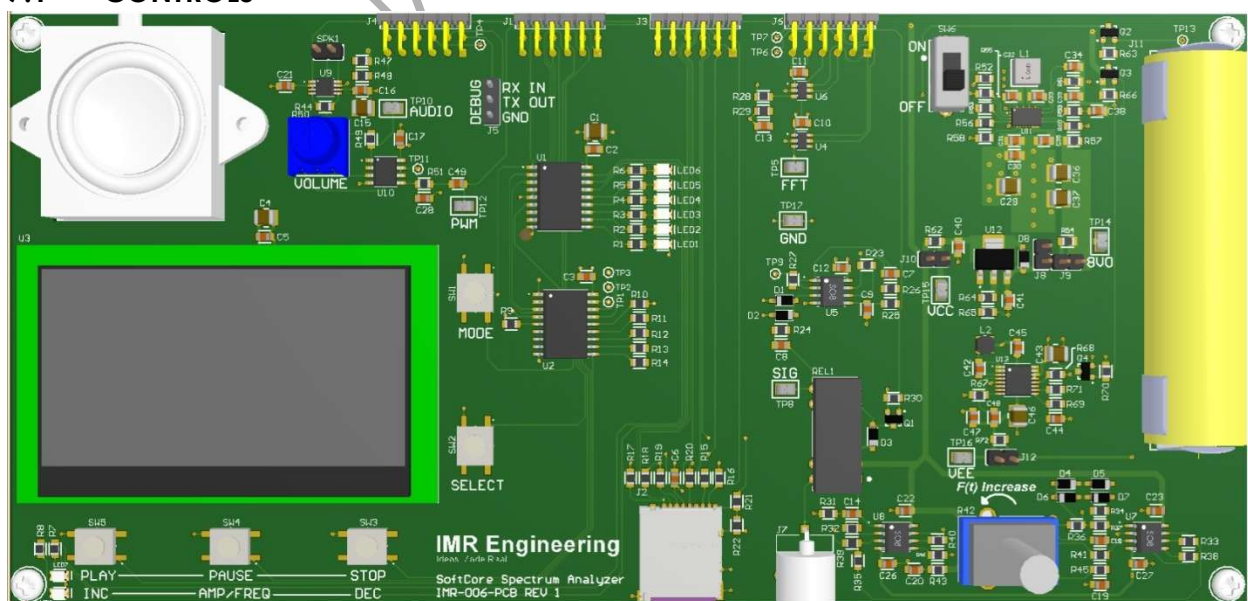
Where:

- V_1 = RMS voltage of the fundamental frequency
- $V_2, V_3, V_4, \dots$ = RMS voltages of the 2nd, 3rd, 4th... harmonics

Not achievable within P1 because of the aliasing effects distorts heavily the calculation, but is intended for phase 2.

7. USER INTERFACE:

7.1 CONTROLS



The UI interface consists of:

AUDIO SA:

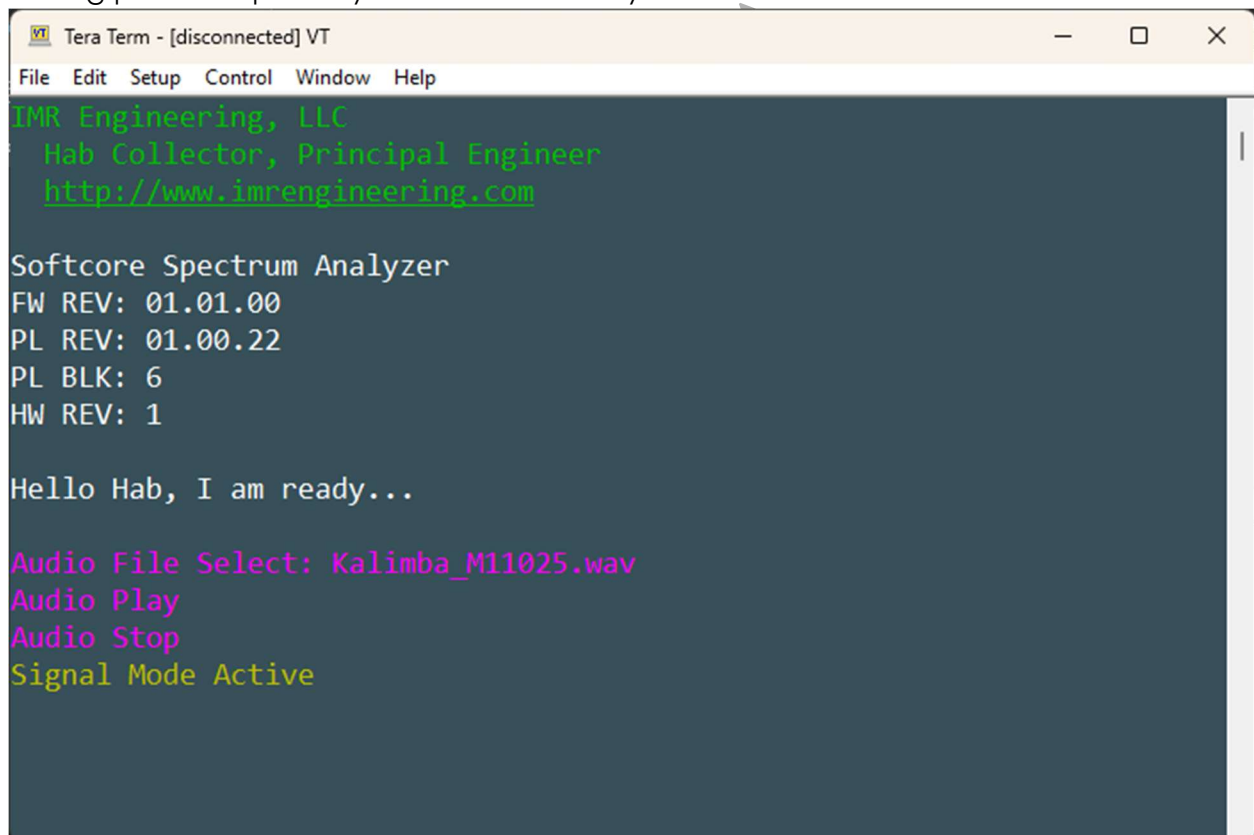
- SW1: MODE: Toggles between Audio and Signal Spectrum Analyzer
- SW2: SELECT: Selects the next audio track
- SW3: STOP: Stop playing audio
- SW4: PAUSE: Pause the present audio playback
- SW5: PLAY: Play the selected track

SIGNAL SA:

- SW1: MODE: Toggles between Audio and Signal Spectrum Analyzer
- SW2: SELECT: Selects between local oscillator and external BNC
- ***As the observable BW is only 25KHz, 1/2 that of the original objective for phase 1, I saw no sense in elaborating on the controls to provide center, and span controls.

7.2 DEBUG PORT:

A serial, logic level, debug port (J5) is added with a standard; N 8 1 1 15200bps interface. The UI is color coded with Audio SA Mode in magenta and Singal SA Mode in Yellow. The debug port is output only for P1. TBD P2 may add user commands.



```
VT Tera Term - [disconnected] VT
File Edit Setup Control Window Help
IMR Engineering, LLC
Hab Collector, Principal Engineer
http://www.imrengineering.com

Softcore Spectrum Analyzer
FW REV: 01.01.00
PL REV: 01.00.22
PL BLK: 6
HW REV: 1

Hello Hab, I am ready...

Audio File Select: Kalimba_M11025.wav
Audio Play
Audio Stop
Signal Mode Active
```

8. BOOT LOADER:

As with everything else, the bootloader process is made more complex with an FPGA. Every FPGA application requires a bootloader. You cannot just OTP program the FLASH.

Unlike a μ C, an FPGA program memory is not limited by the size of on chip FLASH, but rather the size of the external NV memory and DDR. The external memory can be just about anything. For Phase 1 a 16MB on board Quad SPI FLASH is used. That said, it could just as easily interface with the μ SD, with the μ SD providing the flexibility of changing the application just by changing the inserted μ SD card.

An FPGA can facilitate an exceptionally large application; the application size is not limited by the FPGA but by the external NV and attached DDR. In the case of this hardware it is possible to support a 128MB application (DDR limitation). This external FLASH though large, holds far more than the application image. Contained within the external Quad SPI FLASH, and in this order from the base address:

- Hardware Image / Hardware bitstream (BD_Softcore_SA_wrapper.bit)
- Bootloader Image (MB_SSA_BL_App.elf)
- Application Image (MB_SSA_App.srec)

Another key distinction is that a microcontroller HW capabilities are fixed. The FPGA HW capabilities (PL) are completely flexible and loaded at POR, hence the need for the HW image. The HW image will typically be several MB depending on complexity (P1 image was about 3MB). At POR the PL is configured from the hardware bitstream, after PL configuration the bootloader executes from LBM to load the DDR with the application image and the Program counter loaded with the start of the application. Note, if the application were small enough it could have been loaded to LBM. That said, FW functions needed for fast execution were placed in LBM. With the Quad SPI Flash the images are "combined" to a common file .mcs – the .mcs is what is loaded.

The bootloader and the application share the same PL. A separate Quad SPI IP was added to support the external 16MB Quad SPI Flash. That additional interface is not used by the application. Like with a μ C the bootloader is a separate application on to itself.

TBD P2 may instead use the μ SD. The process is essentially the same, the difference being the use of the FAT FS on the μ SD. Though larger and more complex, requiring the FAT FS library and file manipulation. The process is basically the same, just in three separate files: bitstream, bootloader image and application image. Additionally, this bootloader is very slow. It takes about 2 minutes to load. The application image is stored in flash as an .srec file and as such is parsed line by line creating a very slow load. Perhaps for phase 2 I might consider the use of an elf file load – which should drop the boot time to seconds.

9. PHASE 2 LOOK AHEAD:

9.1 ENHANCEMENTS:

In phase two (to come), the focus shifts from "the FPGA can do that too" to "the FPGA can do so much more". Rather than simply replicating μ C peripherals, the design leverages capabilities that are inherently more flexible and impossible via a μ C. Much of the infrastructure created in the first phase is reused and extended, allowing the system

to evolve beyond a microcontroller-style architecture and into a PL hardware-accelerated signal processing platform. Key features of phase 2 include:

- Additional RTL IP:
 - Debounce
 - Async Synchronizer
 - Configurable FIFO with deep memory
 - ADC IP to Interleave ADCs and FIFO for high speed sampling
- Faster clocking of MicroBlaze
- Expanding RAM (currently 128KB expanded to 256KB)
- Signal digital down conversion (DDC: digital mixer + filter + decimation)
- Adjustable Span
- Expanded UI control with scroll wheel
- BW target in MHz range
- DMA

9.2 CORRECTIONS:

- Silk screen add dB rating to sound level bar
- PCB TH test points match size to probe pin
- Remove blocking capacitor at anti-aliasing front end or fix current leakage creating via diodes with no place to go because of blocking capacitor
- Volume pot footprint and orientation both incorrect
- Audio Enable and signal input needs pull-down
- Power supply corrections to: 8V SMPS, -5V SMPS, 5V LDO Regulator
- Anti-aliasing filter -3dB cut-off adjusted
- In Signal SA Mode with external BNC selected but no input drive the display reads the frequency as set by the LO – this shows the sensitivity of the SA as no measurable input is seen by the scope at the AA LPF output. Fixes may be in FW by adjusting the min dB value or more likely in HW disabling the local oscillator and PCB layout changes to avoid coupling.

10. TECHNICAL SPECIFICATIONS:

10.1 ABSOLUTE MAXIMUM RATINGS

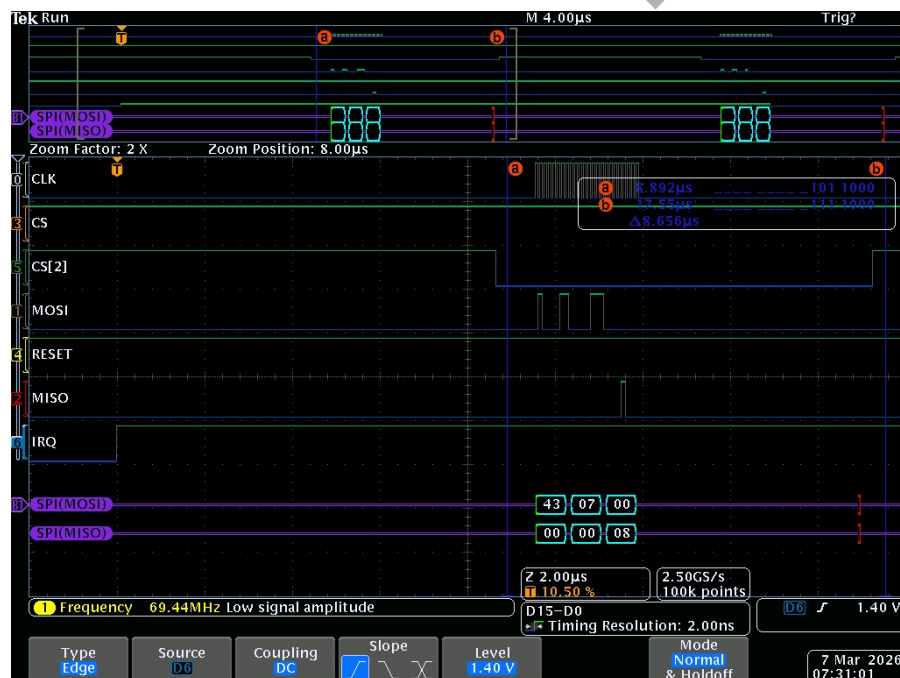
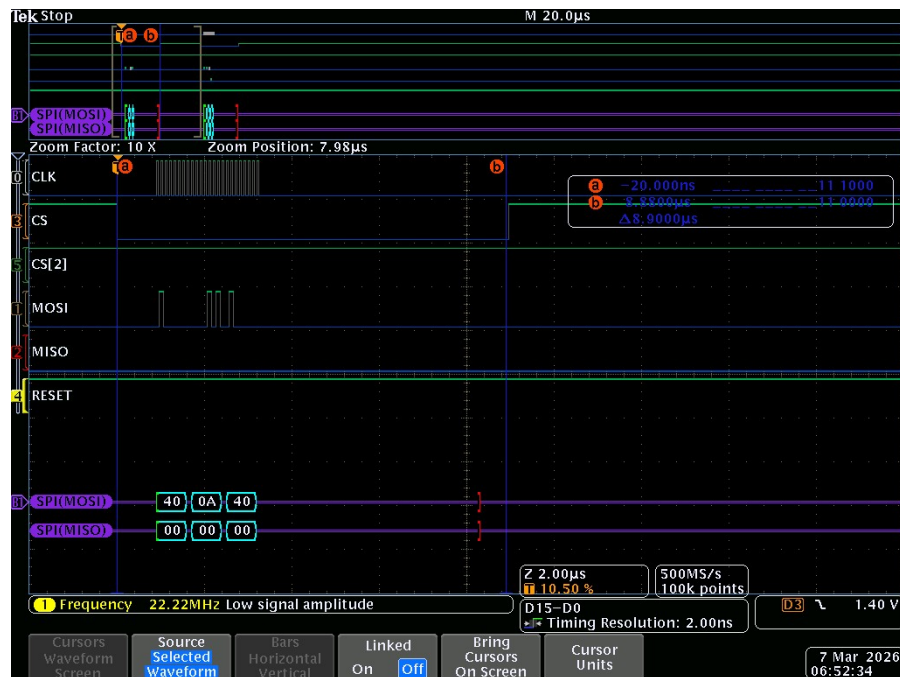
ABSOLUTE MAX RATINGS					
PARAMETER	CONDITION	Min	Typ	Max	UNIT
ENVIRONMENTAL					
Operating Temperature		-20		30	°C
Storage Temperature		-40		85	°C
Relative Humidity	Non-condensing			97	%
SINGLE CELL SUPPLY					
+VS Supply rail		3.4	4.2		V
EXTERNAL BNC INPUT					
Input		-0.5	3.0	3.8	V

10.2 OPERATIONAL RATINGS:

OPERATIONAL SPECIFICATIONS					
PARAMETER	CONDITION	Min	Typ	Max	UNIT
ENVIRONMENTAL					
Temperature		-20		30	°C
Relative Humidity	Non-condensing	0		95	%
PHYSICAL DIMENSIONS					
Length			210.0		mm
Width	Does not include connectors		100.0		mm
Height	Lowest to highest measure wo post		15.0		mm
RUN TIME					
Audio SA mode in playback	3.3V audio amp / 3200mAH cell			160	min
Signal SA or Audio SA (no audio)	3200mAH cell			213	min
AUDIO SA					
WAV File Type	PCM		16		bit
Audio Sample Rate		8	16	22.05	KHz
Max Volume (Measured at .5m)	3.3V Class AB audio amp power			69	dBA
PWM Frequency			100.00		KHz
PWM Audio LPF	-3dB		20.03		KHz
SIGNAL SA					
Input frequency				25	KHz
Input Signal Voltage range	No output loading	0	3	3.1	V
Impedance			1M		Ω
FFT Sample Rate			50		KHz
RBW			48.828		Hz
Aniti-Alasing Filter cut-off	-3dB		329.9		KHz

11. TIMING

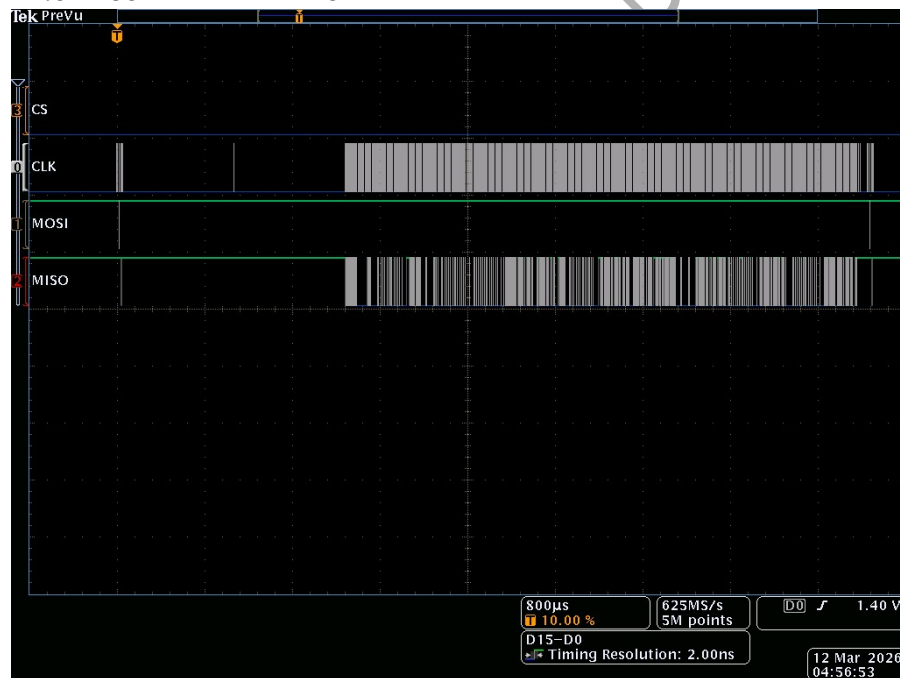
11.1 IO EXPANDERS:



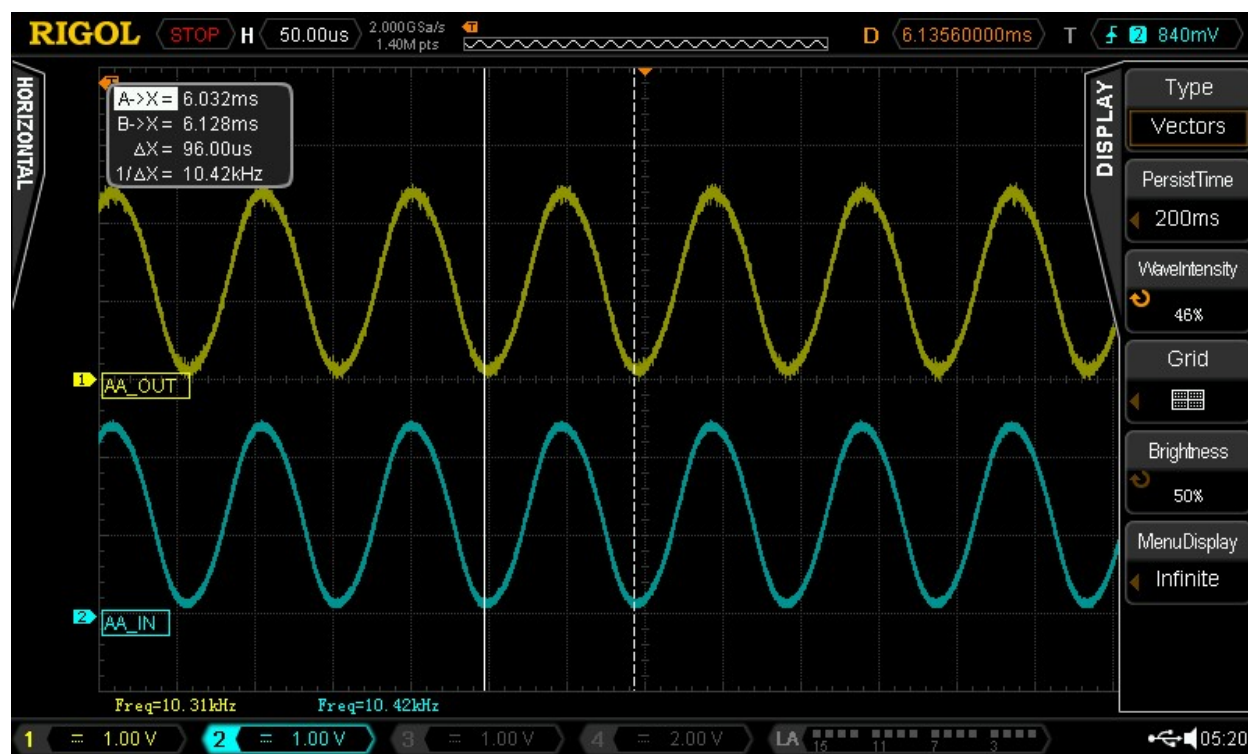
11.2 DISPLAY (SSD1309 CHIP SET):



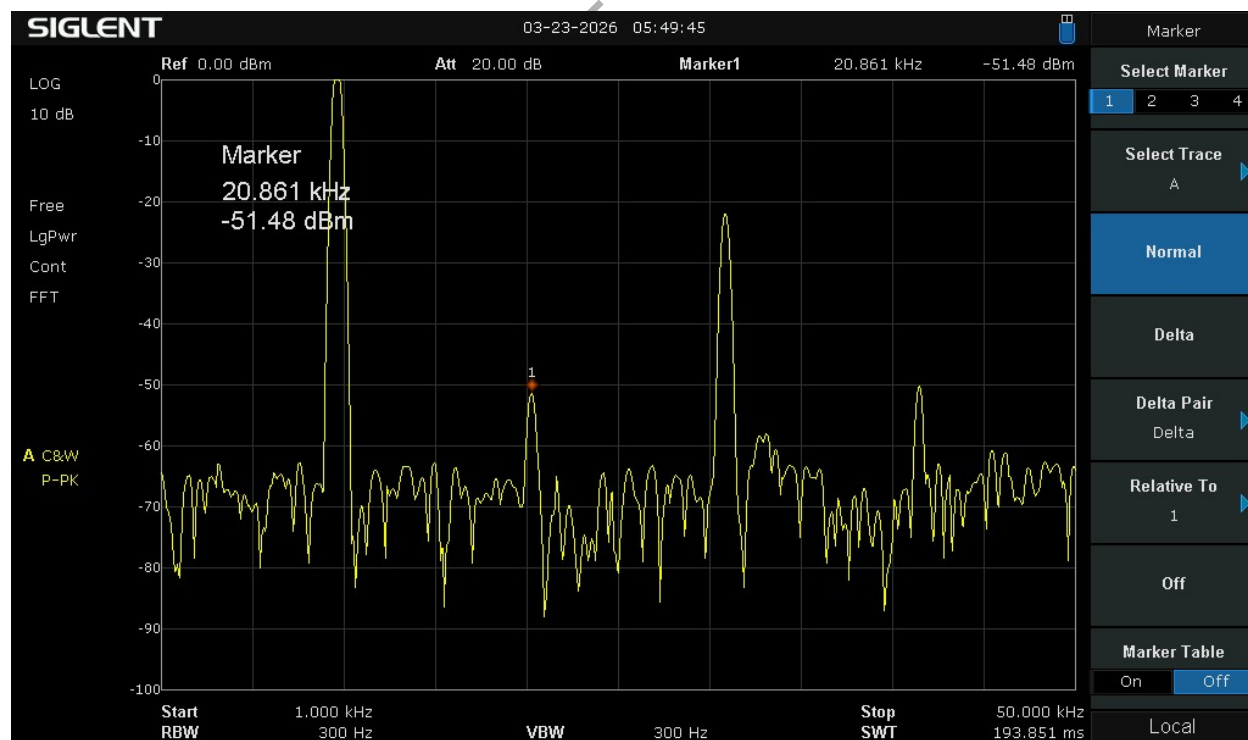
11.3 uSD DATA TRANSFER



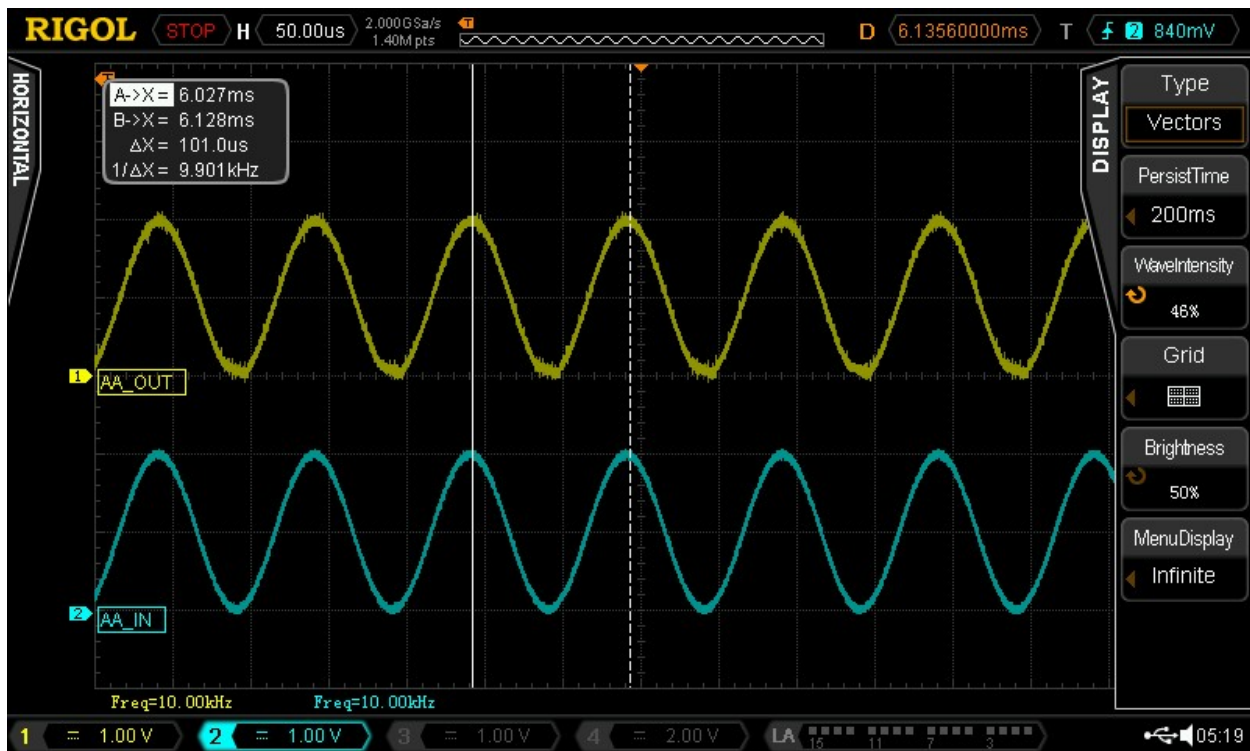
11.4 ANTI-ALIASING FILTER DISCONTINUITY AT THE 0V:



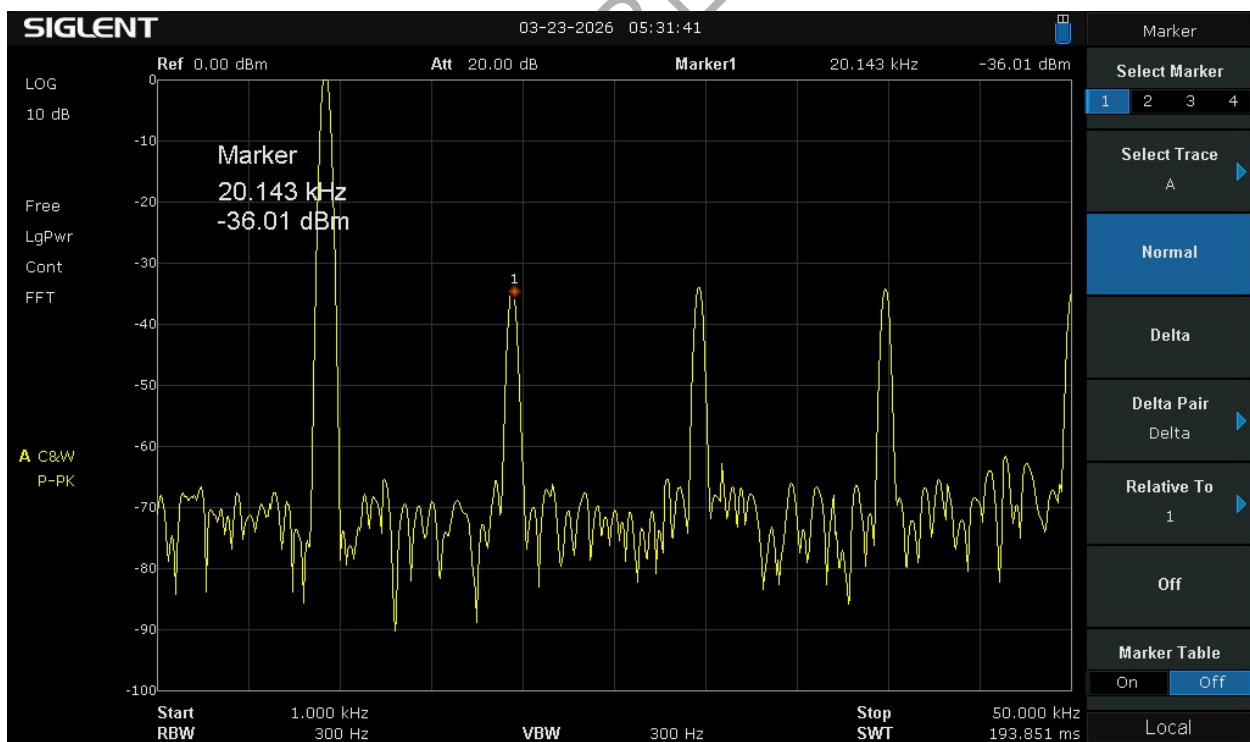
Local Oscillator – Anti-Aliasing filter output – notice no discontinuity around 0V



Local Oscillator – Anti-Aliasing filter output stronger even than odd harmonics



External BNC signal generator input with clipping about the 0V – clipping not present if min signal is raised to 130mV



External BNC signal generator AA LFP output – Strong even and odd harmonics

12. ACKNOWLEDGEMENT

FAMILY:

To the keeper of me, my beloved wife, I share everything with you. You are my strength in weakness, clarity in uncertainty and applause in victory. I accomplish nothing without your support, nor is anything of worth unless I can share it with you. I love you.

To my daughter Christina, my first born, we have matured and grown together. You are wonderful in spirit, a lover of children, and you put your heart into everything you do. Love is what you do. You are exceptional in so many ways; add to your gifts temperament and discernment and the world is yours.

To my daughter Faith, the Humanitarian, it is clear that you belong to the goodness of this world. You must remember that true greatness is only realized through service and that there are many ways to serve. Therein will be your pursuit – run to catch it. I hope I have shown you how to run.

To my daughter Trinity, the engineer, there is a clarity in the way you speak and think, in your questions and answers. You are to add to the scientific collective. Of the three, you are most like me. Don't be afraid, nor ashamed of whom you are; accept it, and work hard to be.

To God Almighty, it is you that have made me, and not me myself – thank you for the many blessings, may I be a better servant and steward.

FRIENDS & COLLEAGUES:

Special thanks to my friend Walter Varda, Raytheon an absolute Rock Star of analog engineering and an even better person.

Special thanks to Timoth Brothers, Professor Electrical Engineering, GA Tech, a great sound board for crazy ideas and limiting project scope. Thank you for teaching and inspiring Trinity, we are forever grateful.